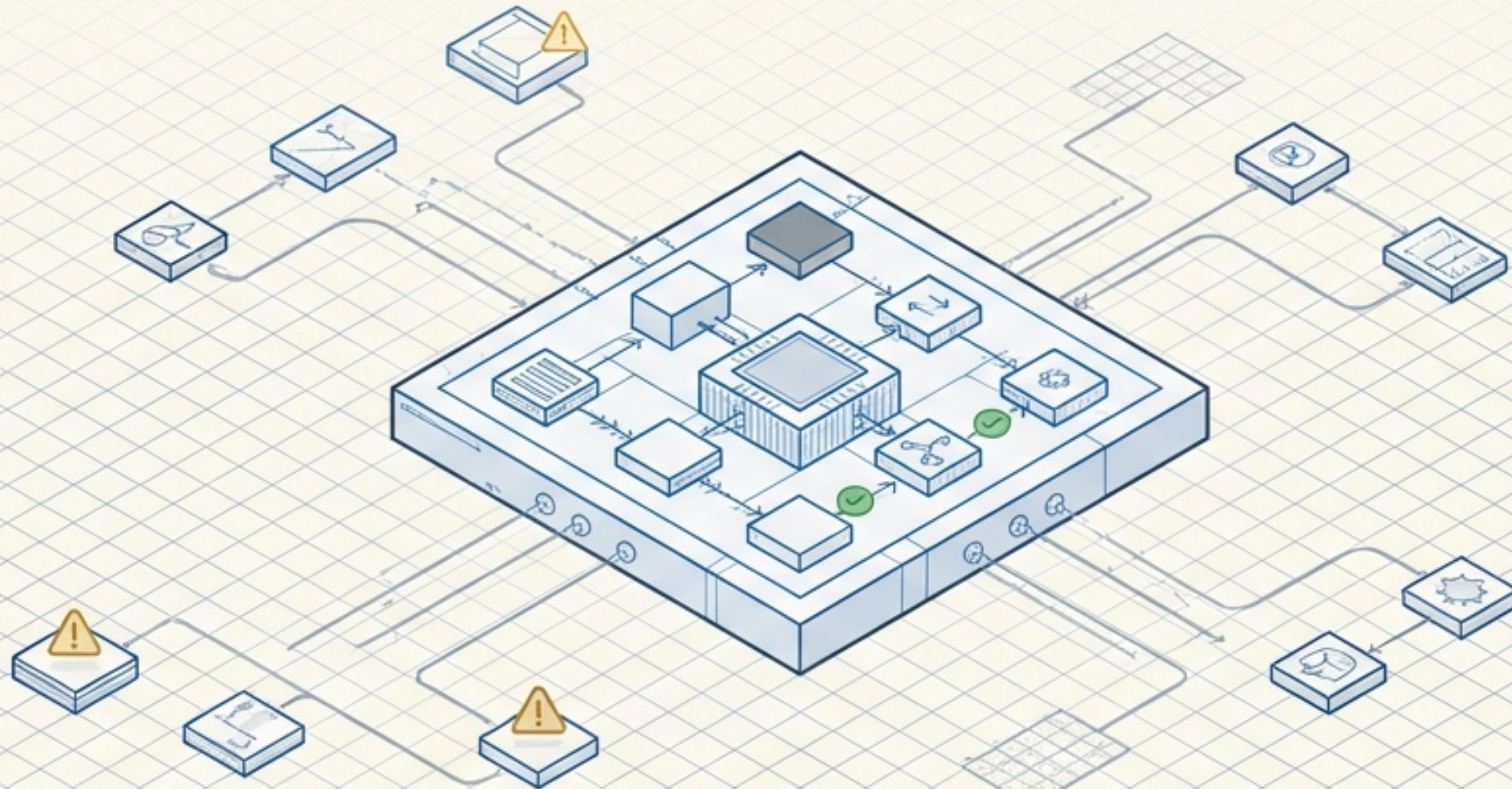


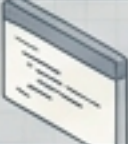
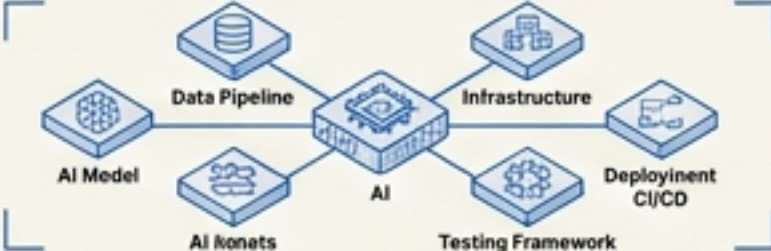
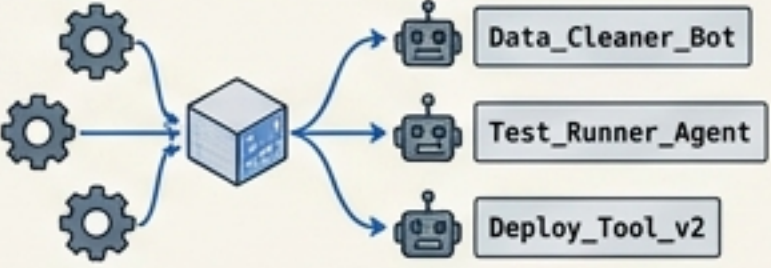
HARNESSE ENGINEERING

29 Precepts to Build the Systems That Build Software

> Stop writing code. Start engineering the environment.

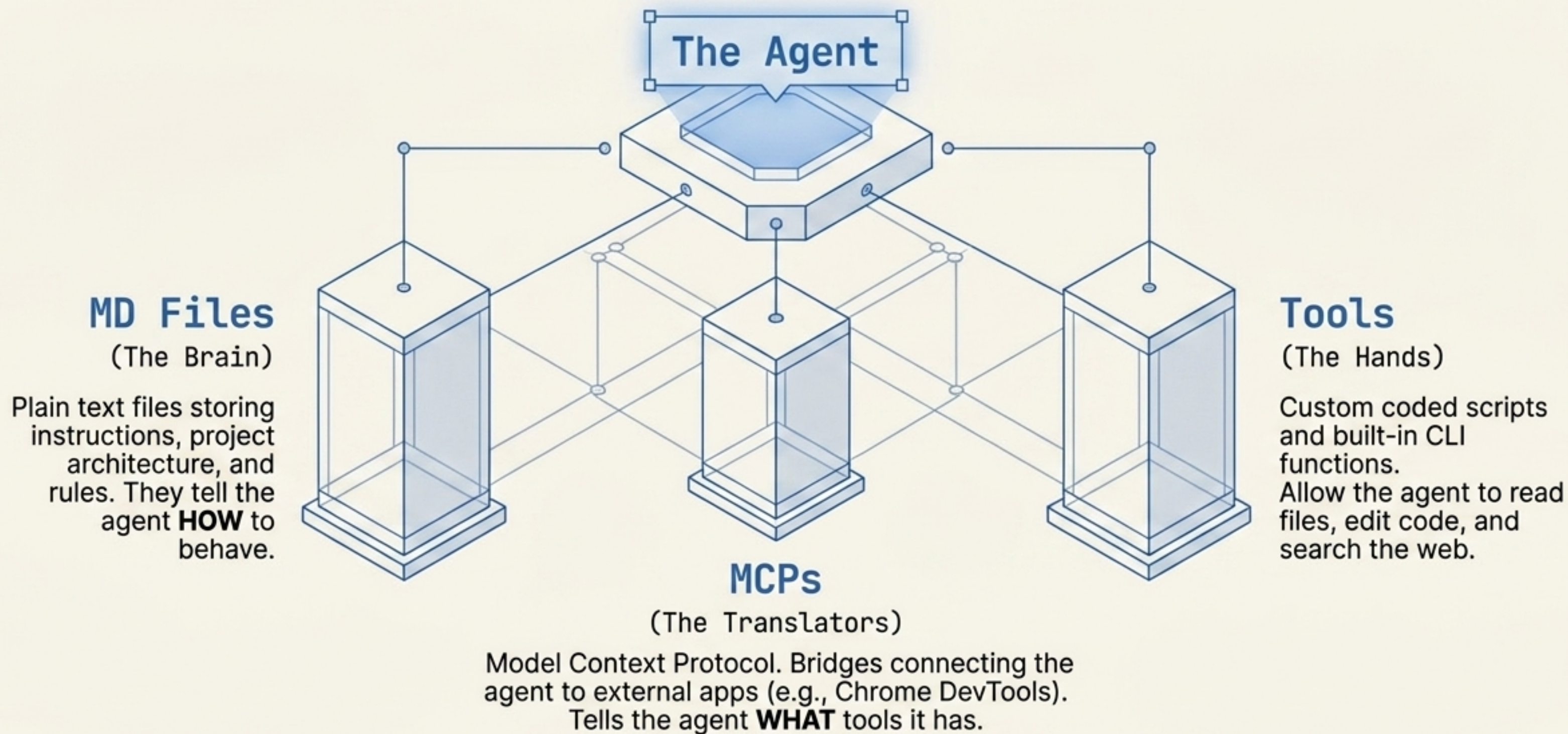


The Developer is Dead. Long Live the Harness Engineer.

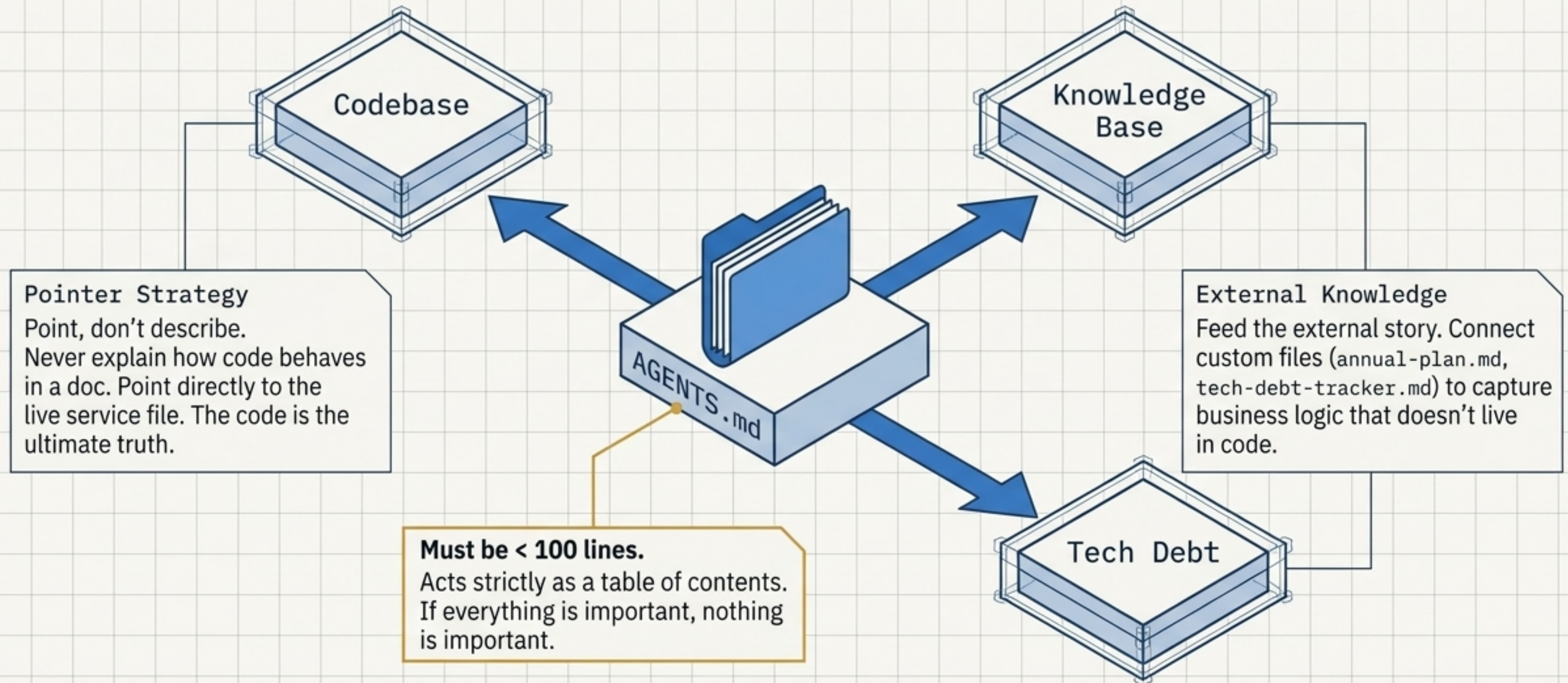
	The Typist	The Architect
Role	Writing code line-by-line. 	Building the environment around the AI. 
Fixing Mistakes	Correcting the code quietly by hand. 	Updating instructions so the mistake is gone for good. 
Planning	Keeping plans in your head or transient chats. 	Specs Driven Development (SPECS.md). 
Tooling	Depending on 3rd-party SaaS apps. 	Spawning agents that build custom tools on the fly. 

If you only fix the code, the same mistake comes back tomorrow.
Fix the instructions, and it's gone for good.

Anatomy of an Agentic Environment

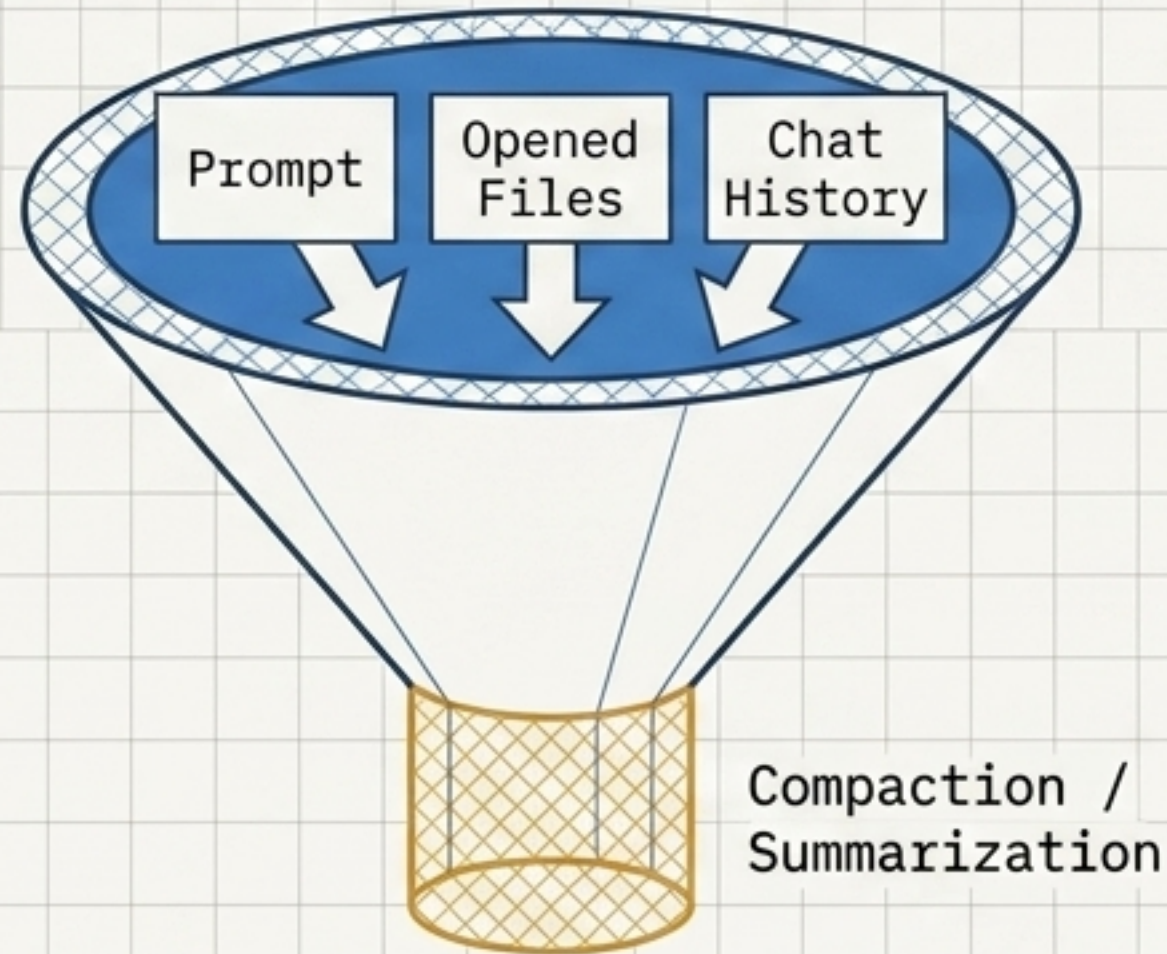


Design Your Docs Like a Router, Not an Encyclopedia



The Agent's Memory is Fragile

The Context Compactor



The agent starts every chat empty.

Once the context window fills, the chat compresses, details fade, and the agent begins hallucinating project rules.

Pro-Tip

Spawn Subagents.

Instruct the main agent to spawn helper subagents to deep-dive into databases or code search.

Helpers return only a concise summary, keeping the main builder's context clean.

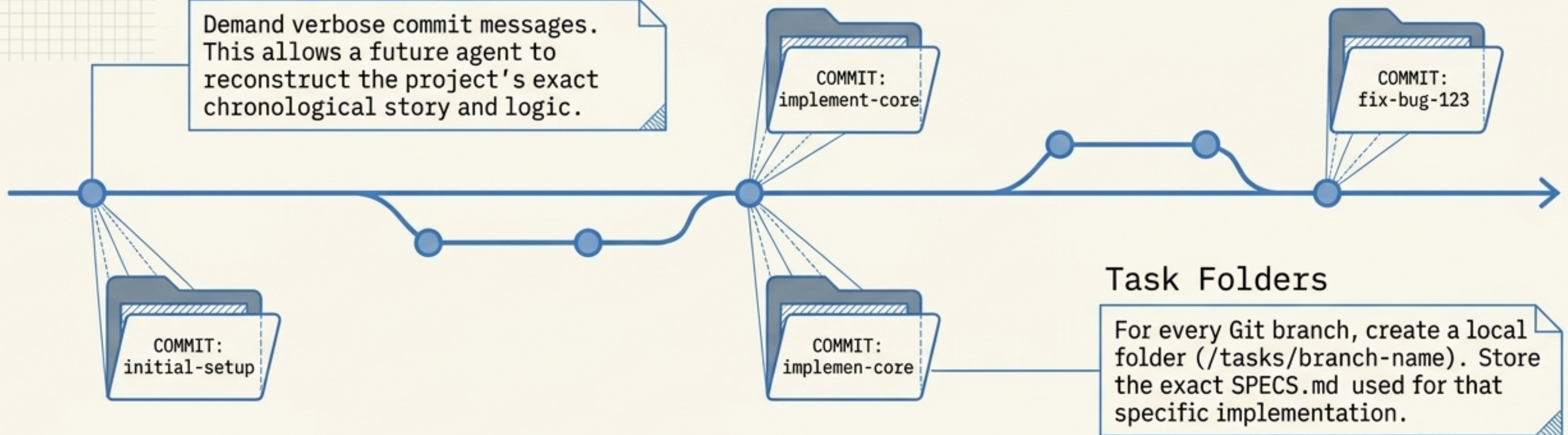
The Golden Rule

One task, one fresh chat.
Never fall in love with your chat history.

Transfer Memory from Chat to File System

Detailed Commits

Demand verbose commit messages. This allows a future agent to reconstruct the project's exact chronological story and logic.



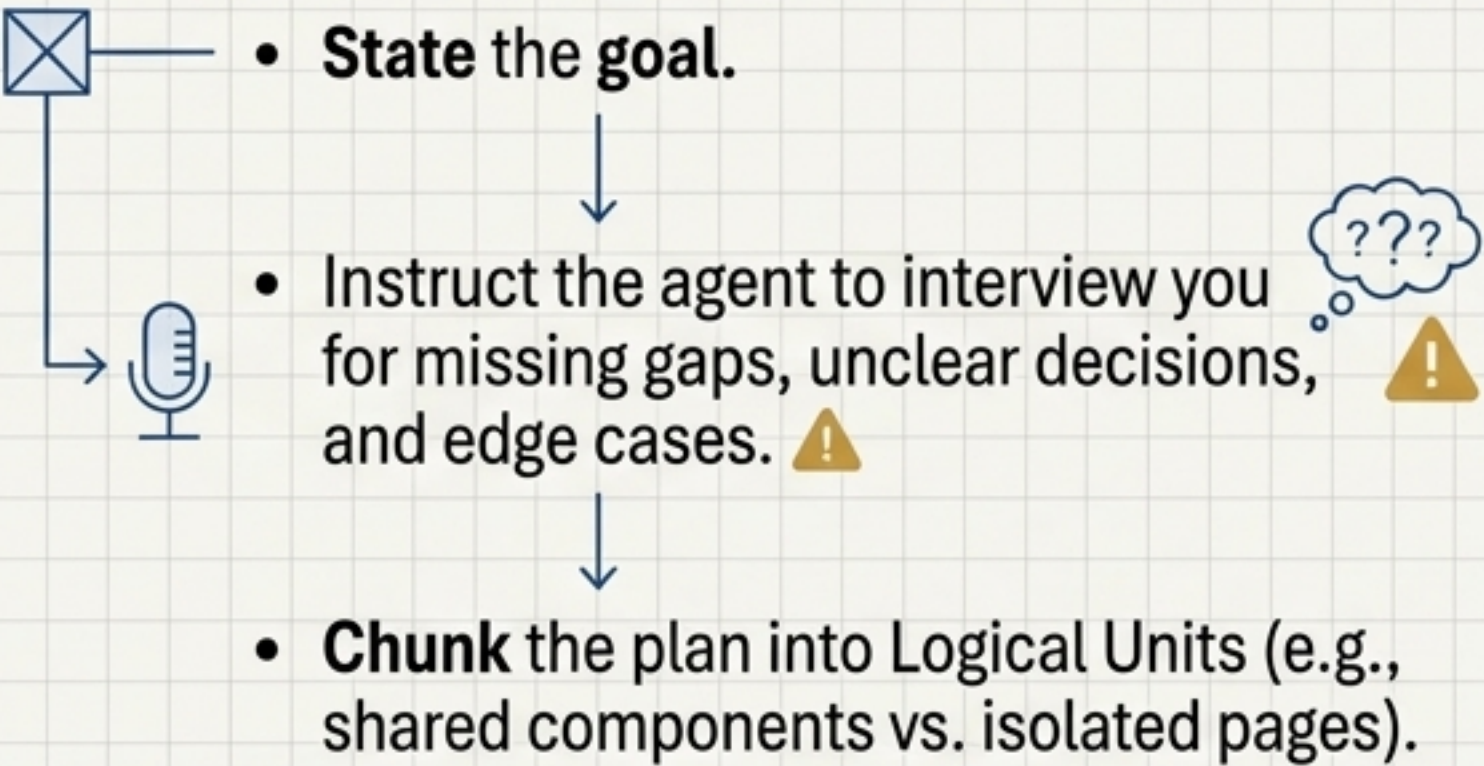
The Result

An agent can jump into a 5-year-old bug, trace the Git history chronologically, read original specs, and uncover forgotten architectural decisions.

Never Plan and Build in the Same Step

The Interview

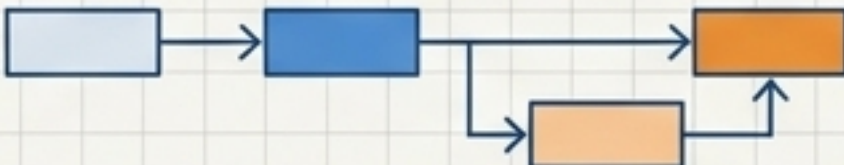
(representing the human-to-AI planning phase)



The Output

 SPECS.md

- A living markdown checklist.
- Ordered chronologically for reading, but executed by **shared importance**.


- The agent ticks [x] boxes when a logical unit is done, anchoring its location.

 Implement Header Component (Shared)

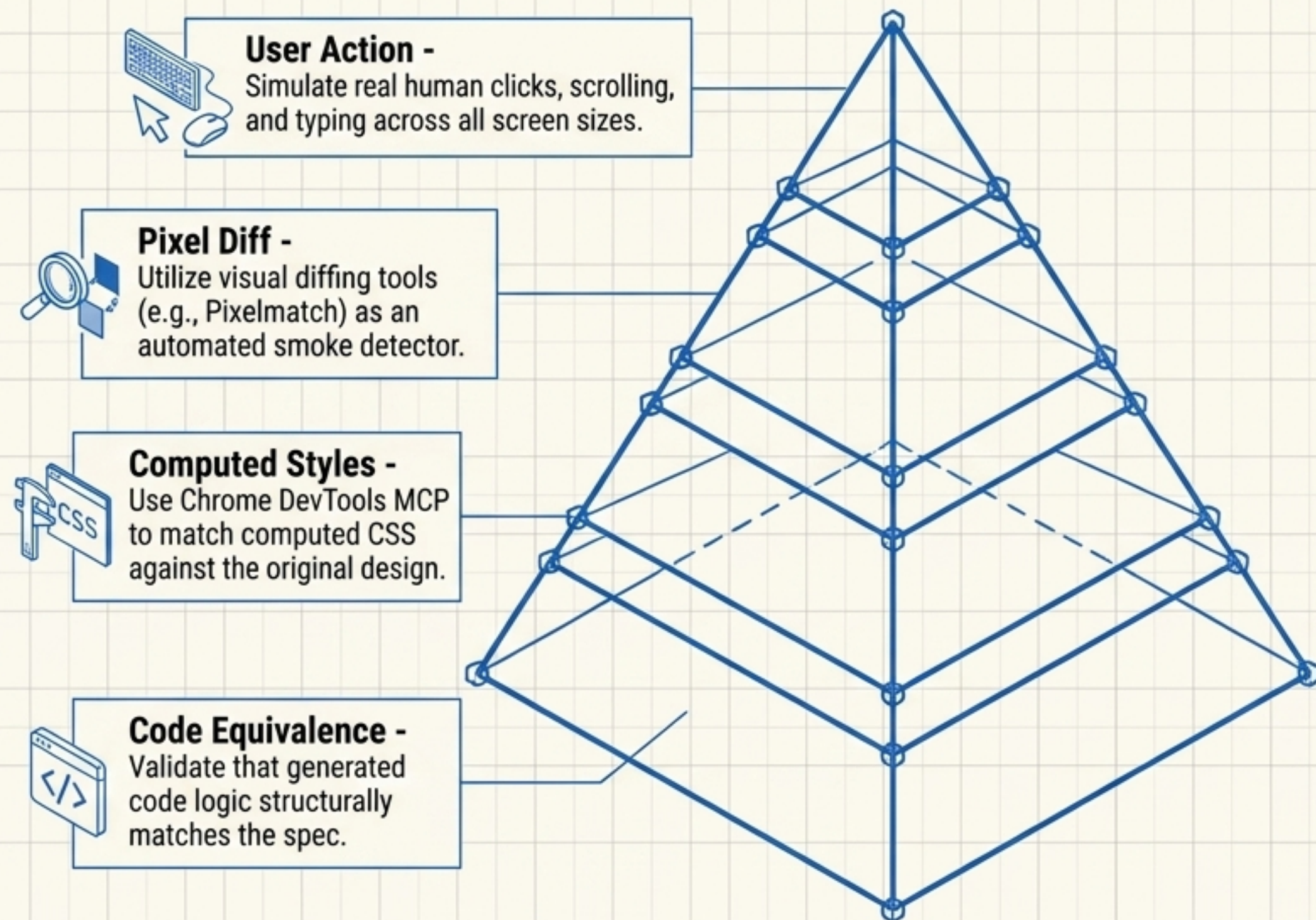
 

 Create Login Page (Page A)



 Develop Dashboard (Page B)

Done is Observable. Never Accept a Feeling.



100% test coverage
is the new standard.

Tests are the fences
that let agents run
fast without
breaking things.

The Engine of Autonomy: The Ralph Loop



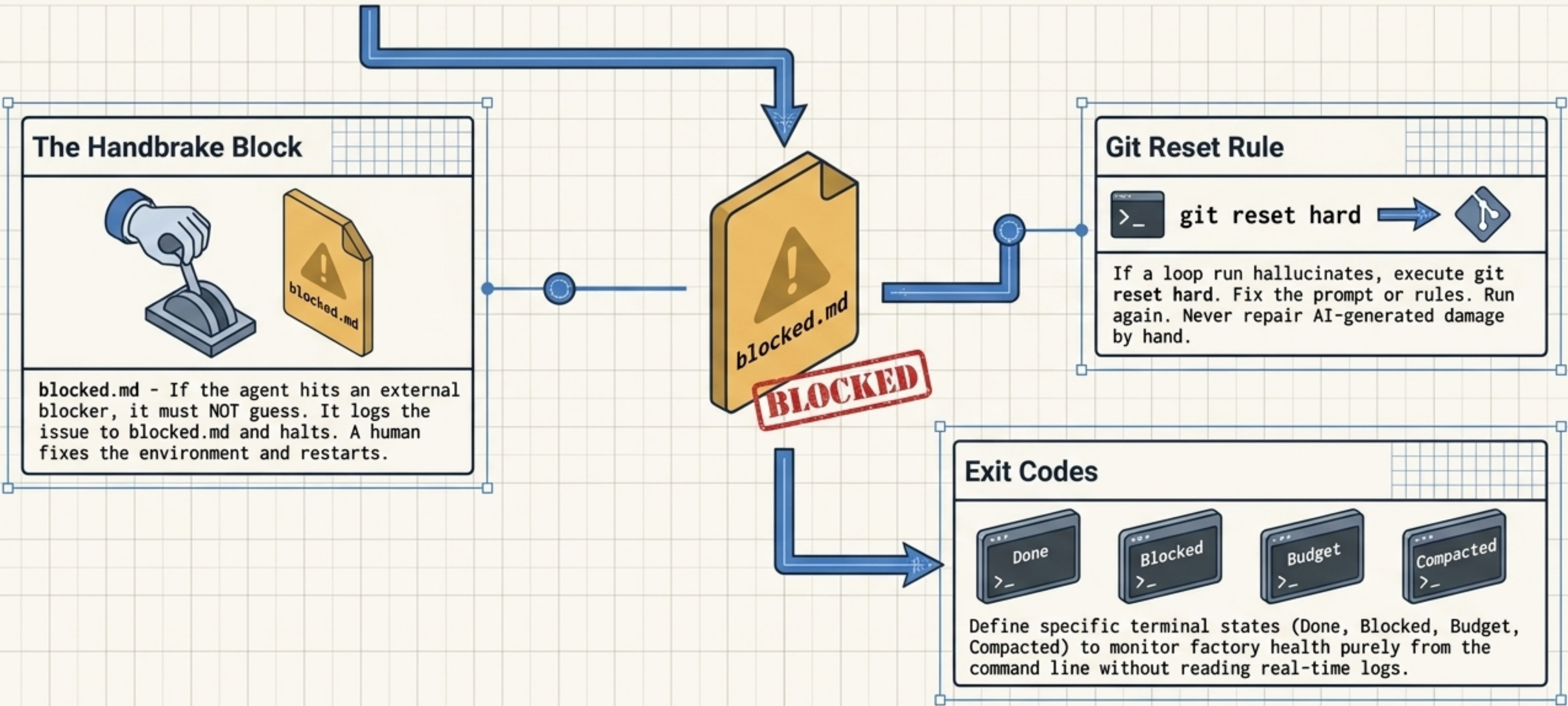
Mechanism

Place your prompt into a file and run the `CLI agent` in a non-interactive bash loop. The agent acts entirely on its own with a fresh context window for every item.

Exit Condition

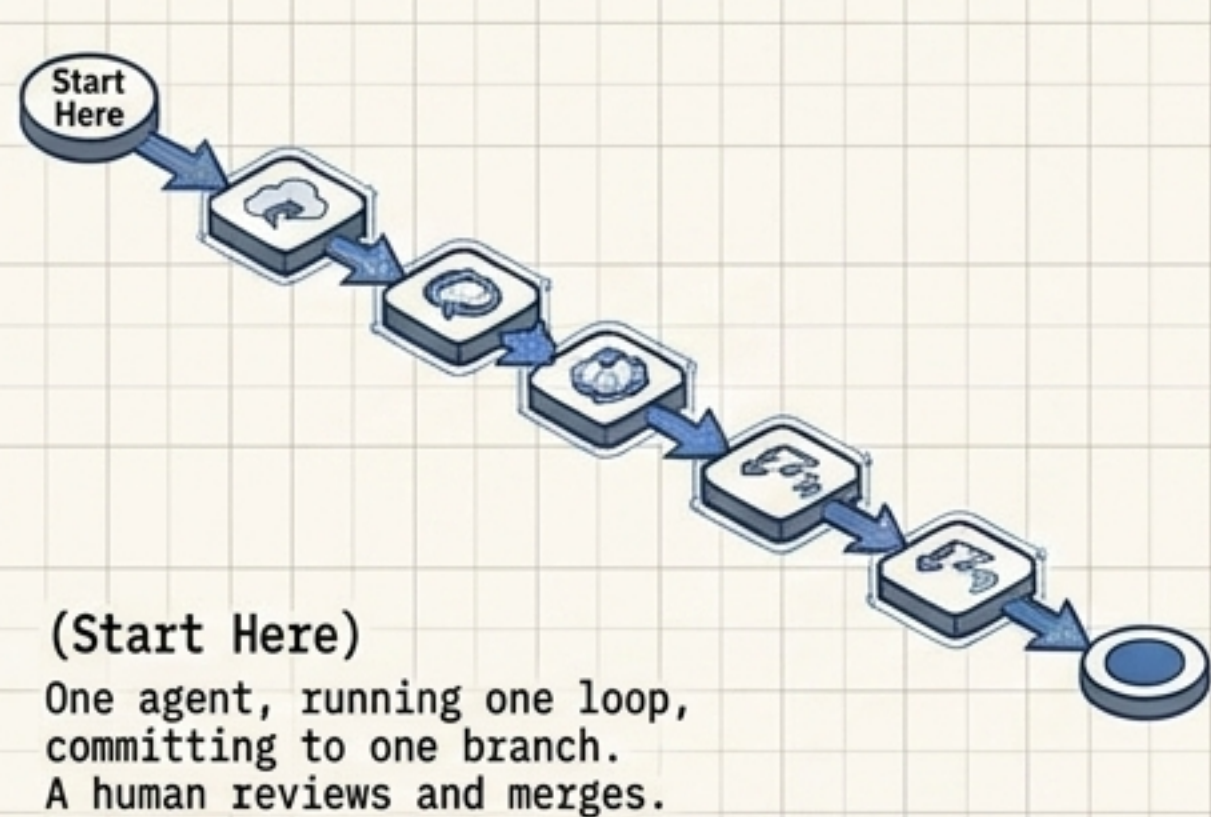
The loop automatically terminates when zero unticked boxes remain in `SPECS.md`.

Handling Loop Failures Without Manual Fixes

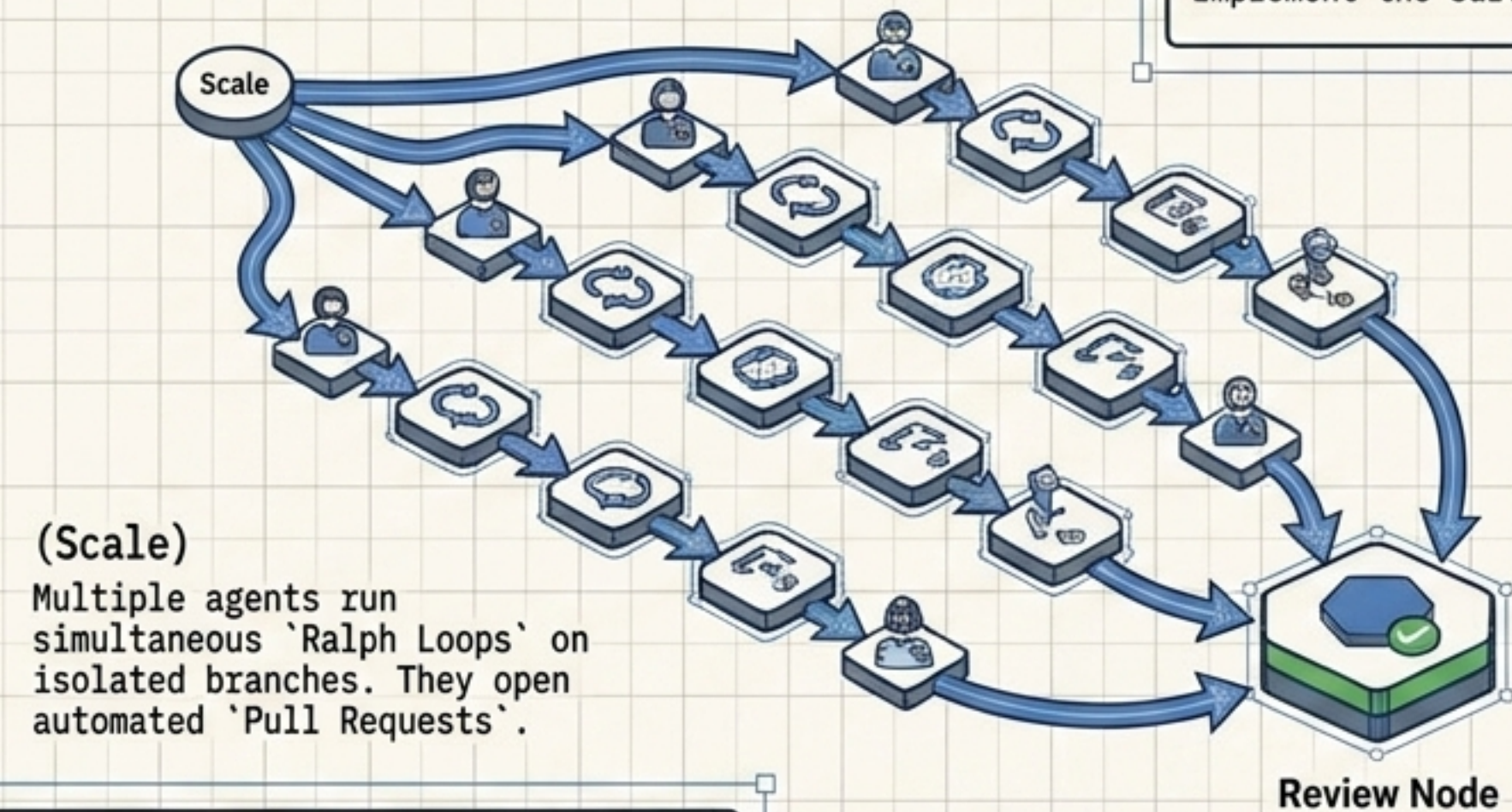


From Solo Agent to Autonomous Teams

Sequential



Parallel



💡 Pro-Tip Box

Design Brawls
Spawn two agents to argue about architecture. One proposes, the other attacks. Implement the survivor.

Reviewer Agent

A high-context Senior Dev agent watches the 'repo', reviews 'PRs', resolves merge conflicts by comparing logic, and merges safe code.

The Harness Evolution Ladder

